

Load Test Report: Test 2 of 3, 100 Concurrent Users

Date: 26 Sep 2026, 10:15 – 10:21 IST
Target: `https://api.iverto.ai/devhoste1`, Hostel **v2** API (`/v2/tenants/acme/...`)
Tool: k6 v2.2.0, `k6/v2/users.js`, run from one Windows laptop (one public IP)
Traffic: reads only. No passes, decisions or other writes were sent.

1. Verdict

FAIL (latency). The server did not fail outright. It just got slow for everyone.

SLO	Target	Result	
HTTP error rate	< 1%	0.69% (64 / 9,252)	✓
Checks passed	> 99%	99.30%	✓
Latency p95	< 800 ms	5.78 s (7× over)	✗
Latency p99	< 2 s	8.63 s	✗
Median latency		2.01 s (was 118 ms at 30 users)	✗
GET <code>/dashboard</code> success	100%	2% (52 of 53 returned 500)	✗
GET <code>/roster</code> success	100%	12% (7 of 8 timed out at 30 s)	✗
GET <code>/groups</code> success	100%	0% (5 of 5 returned 500)	✗

The error rate stays under 1% only because 97% of traffic is cheap student reads, and they queue rather than fail. **A student opening the app now waits ~28 s** (p50 journey time, including ~5 s of think time) instead of ~9 s.

2. Test setup

Item	Value
Model	Closed: each virtual user opens the app (one role journey), idles 3–8 s, repeats
Profile	0 → 100 users over 1 min · hold 100 for 4 min · ramp down 30 s (total 5 m 40 s)
Role mix	student 80% · guardian 15% · warden 4% · admin 1%
Token pool	35 tokens (24 / 6 / 3 / 2), logged in 10:11–10:15, 0 × 429
Request timeout	30 s

3. Results

3.1 Totals, compared with Test 1

Metric	30 users	100 users	Change
Requests served	8,029	9,252	+ 15%
Throughput (average)	23.8 req/s	27.2 req/s	+ 14%

Metric	30 users	100 users	Change
Journeys per second	2.65	3.03	+ 14%
Latency p50	118 ms	2.01 s	17×
Latency p95	636 ms	5.78 s	9×
Latency p99	1.26 s	8.63 s	7×
Journey duration p50 / p95	9.2 s / 14.1 s	27.5 s / 47.1 s	3×
Failed requests	29	64	

3.3× more users produced only 14% more served traffic. The extra users just waited in line. This confirms a hard server-side ceiling of **~27–30 req/s**.

3.2 Timeline (15 s buckets)

Time	Users	req/s	p50	p95	Errors
0–15 s	ramp	13.5	119 ms	451 ms	0%
15–45 s	ramp ~25 → 75	30 – 40	0.4 – 1.8 s	1.4 – 2.8 s	0.3 – 0.8%
45–90 s	75 → 100	25 – 31	2.2 – 3.0 s	3.0 – 4.1 s	0.5 – 0.7%
90–105 s	100	11.9	7.3 s	9.3 s	3.4%
105–180 s	100	25 – 38	1.8 – 3.3 s	2.8 – 5.7 s	0.2 – 1.1%
180–240 s	100	16 – 21	3.2 – 4.8 s	5.8 – 8.4 s	1.6 – 1.8%
240–315 s	100 → ramp down	24 – 42	1.6 – 3.4 s	2.4 – 6.8 s	0 – 0.6%

Throughput **peaked at ~40 req/s for a few seconds**, then fell back to ~27. Three times (90 s, 180 s, 210–225 s) it collapsed to 12–18 req/s with p50 of 5–7 s. Each time lines up with a burst of `/dashboard`, `/groups` or `/roster` calls holding DB connections.

3.3 Per endpoint (sorted by p95)

Endpoint	Calls	p50	p95	Max	Errors
GET <code>/roster</code>	8	30 s	30 s	30 s	7 × timeout (88%)
GET <code>/groups</code>	5	17.2 s	22.6 s	22.6 s	5 × 500 (100%)
GET <code>/dashboard</code>	53	16.3 s	20.7 s	22.7 s	52 × 500 (98%)
GET <code>/memberships</code>	3	6.6 s	11.2 s		0
GET <code>/me/children/:id</code>	38	3.0 s	8.2 s	9.5 s	0
GET <code>/me/student</code>	827	2.6 s	7.1 s	9.3 s	0
GET <code>/me/children</code>	150	2.9 s	6.2 s	9.4 s	0
GET <code>/sites</code>	880	2.2 s	6.2 s	9.0 s	0
GET <code>/me/curfew</code>	827	2.4 s	6.1 s	8.6 s	0
GET <code>/me/permissions</code>	1,190	1.8 s	5.7 s	9.0 s	0
GET <code>/me/branding</code>	1,030	2.1 s	5.1 s	8.9 s	0
GET <code>/app-config</code>	1,030	1.8 s	4.4 s	8.4 s	0

Endpoint	Calls	p50	p95	Max	Errors
GET /me/notifications/unread-count	1,030	1.7 s	4.3 s	8.9 s	0

Even `/app-config`, a constant lookup that served ~194 req/s alone on 24 Sep, now has a **1.8 s median**. It isn't slow in itself. It is waiting behind the same saturated resource as everything else (DB pool / connection queue).

4. What went wrong

- Hard throughput ceiling at ~27–30 req/s.** Going from 30 to 100 users added 14% throughput and multiplied latency 9–17×. The backend is fully saturated. Any further users only lengthen the queue.
- Every endpoint became slow, including trivial ones.** `/app-config` p50 went 95 ms → 1.8 s, which points to a **shared bottleneck** (DB connection pool, or one Node process whose event loop is waiting on DB callbacks), not one bad query.
- Staff screens now fail almost completely.** `/dashboard` 98% 500, `/groups` 100% 500 (it was 0% errors at 30 users), `/roster` 88% timeouts. The server returns 500 after ~15–22 s, which fits a statement/acquire timeout.
- Periodic stalls.** Three windows where throughput fell to 12–18 req/s and p95 went to 8–9 s, each caused by heavy staff queries piling up.
- Low error rate hides the problem.** Real users won't wait 28 s. They will force-close and reopen the app, which adds even more requests (retry amplification). This test didn't model that.

5. Fixes

#	Problem	Fix
1	~30 req/s ceiling, all endpoints queue	Find the saturated resource first: pull server metrics for 10:15–10:21 IST (DB pool wait time, active connections, CPU, event-loop lag). Most likely the Postgres pool (small default size, e.g. 10) held by slow staff queries.
2	Slow queries holding the pool	EXPLAIN (ANALYZE, BUFFERS) on <code>/dashboard</code> , <code>/roster</code> , <code>/groups</code> , <code>/me/student</code> , <code>/sites</code> , <code>/me/curfew</code> . Add composite indexes: <code>(tenant_id, site_id)</code> , <code>(tenant_id, student_id)</code> , <code>(tenant_id, status, created_at)</code> on <code>permissions</code> , <code>(student_id)</code> on <code>guardian links</code> , <code>(group_id)</code> on <code>group members</code> .
3	<code>/dashboard</code> / <code>/groups</code> / <code>/roster</code> 98–100% failure	One aggregate query + 15–30 s cache for dashboard. DB-side pagination for roster. Remove N+1 in groups. Target < 300 ms each.
4	Staff reports starve students	Separate pool (or read replica) for reports. <code>statement_timeout</code> ≈ 5 s.
5	Cheap endpoints wait behind DB	Cache <code>/app-config</code> , <code>/me/branding</code> , <code>/sites</code> , <code>/categories</code> in memory or Redis (they change rarely), with <code>Cache-Control</code> / <code>ETag</code> so the app can skip the call. That removes ~35% of all requests from the DB path.
6	Queue grows without bound	Explicit pool size (20–30), 1–2 s acquire timeout, return a fast 503 + <code>Retry-After</code> when saturated, so the app backs off instead of users retrying on a 30 s spinner.
7	Single instance	After the query fixes, run ≥ 2 app instances behind Caddy with active health checks.

6. Raw files

`load-test/results/2026-09-26/users-100.{csv,json,log}`, `users-100.csv.analysis.json`, `pool-100.log`.
 Reproduce: `node scripts/login-pool-v2.mjs && k6 run -e USERS=100 -e HOLD=4m k6/v2/users.js`